CS3230

AY25/26 Sem 1

by ngmh

Asymptotic Analysis

Recursive Fibonacci

- Let $T(n)$ be the number of operations done by $Fib(n)$
- $T(0) = T(1) = 2$ for checking if statement and return
- $\forall n \geq 2, T(n) = T(n-1) + T(n-2) + 8$ for the extra checking and calls
- So $T(n) \geq Fib(n)$
- $Fib(n) = Fib(n-1) + Fib(n-2), Fib(n) \geq 2 \cdot Fib(n-2)$
- Doubles every two terms, so it doubles $\lceil \frac{n-2}{2} \rceil$ times and $T(n) \geq Fib(n) \geq 2^{\frac{n-2}{2}}$

Big O Notation

- $f \in O(g) : \exists c > 0, n_0 > 0, \forall n \geq n_0, 0 \leq f(n) \leq cg(n)$, g is an upper bound on f, also written as $f(n) = O(g(n))$
- Multiple c and n_0 could work for the same case
- $f \in \Omega(g) : \exists c > 0, n_0 > 0, \forall n \geq n_0, 0 \leq cg(n) \leq f(n)$, g is a lower bound on f
- $f \in \Theta(g)$: tight bound, $\Theta(g) = O(g) \cap \Omega(g)$
- $f \in o(g)$: $\forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) < cg(n)$, strict upper bound
- $f \in \omega(g)$: $\forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq cg(n) < f(n)$, strict lower bound

Limits

- $lim_{n\to\infty}(\frac{f(n)}{g(n)}) = 0 \Rightarrow f(n) \in o(g(n))$
- $lim_{n\to\infty}(\frac{f(n)}{g(n)}) < \infty \Rightarrow f(n) \in O(g(n))$
- $0 < lim_{n\to\infty}(\frac{f(n)}{g(n)}) < \infty \Rightarrow f(n) \in \Theta(g(n))$
- $lim_{n\to\infty}(\frac{f(n)}{g(n)}) > 0 \Rightarrow f(n) \in \Omega(g(n))$
- $lim_{n\to\infty}(\frac{f(n)}{g(n)}) = \infty \Rightarrow f(n) \in \omega(g(n))$

Example Limit Proof

- By definition, $lim_{n\to\infty}\frac{f(n)}{g(n)} = 0$ means $\forall \varepsilon > 0, \exists n_0 > 0, \forall n \geq n_0, \frac{f(n)}{g(n)} < \varepsilon$.
- We can pick $c = \varepsilon$, then we have $f(n) < \varepsilon \cdot g(n), f(n) < c \cdot g(n), f(n) \in o(g(n))$.

Properties

1. Reflexivity of O, Ω, Θ: $f(n) \in O(f(n))$
2. Transitivity of all: $f(n) \in O(g(n)) \wedge g(n) \in (h(n)) \Rightarrow f(n) \in O(h(n))$
3. Symmetry: $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$
4. Complementarity: $f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$, $f(n) \in o(g(n)) \Leftrightarrow g(n) \in \omega(f(n))$

Useful Facts

- $e^x \geq 1 + x$
- For any $k > 0, a > 1$, $n^k \in o(a^n)$
- $n^i \in \Omega(log^j n)$
- $log_b a = \frac{log_c a}{log_c b}$
- $log_b a = \frac{1}{log_a b}$
- $a^{log_b c} = c^{log_b a}$
- Stirling's Approximation: $n! = \sqrt{2\pi n}(\frac{n}{e})^n(1 + \Theta(\frac{1}{n}))$, $log(n!) \in \Theta(n\ log\ n)$
- $\sum_{k=1}^{n} k = \frac{1}{2}n(n+1) \in \Theta(n^2)$
- $\sum_{k=0}^{n} x^k = \frac{x^{n+1}-1}{x-1}$
- $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$ when $|x| < 1$
- $H_n = \sum_{k=1}^{n} \frac{1}{k} = ln\ n + O(1)$
- L'Hopital's: $lim_{x\to\infty}\frac{f(x)}{g(x)} = lim_{x\to\infty}\frac{f'(x)}{g'(x)}$
- Conditional Probability: $P(A|B) = \frac{P(A \cap B)}{P(B)}$
- Baye's Theorem: $P(A|B) = \frac{P(A)P(B|A)}{P(B)} = \frac{P(A)P(B|A)}{P(A)P(B|A)+P(\overline{A})P(B|\overline{A})}$
- For constant i, $a > 0$, $(n+a)^i \in O(n^i)$

WA1 Asymptotics

- $f(n) = 3^n, g(n) = 2^n \cdot n^{10000}$, $f(n) \in \omega(g(n))$
- $f(n) = 2^{(2^n)}, g(n) = 2^{(2^{n+1})}$, $f(n) \in o(g(n))$ since $g(n) = (f(n))^2$
- $f(n) = \sum_{i=1}^{n} \frac{1}{2^i}, g(n) = \sum_{i=1}^{n} \frac{i^{10000}}{2^i}$, $f(n) \in \Theta(g(n))$ since exponential grows faster than polynomial
- $f(n) = log\ log\ n^n, g(n) = log\ log\ 2^{\sqrt{n}}$, $f(n) \in \Theta(g(n))$, since both are exponential in polynomial in n, double logarithm is $\Theta(log\ n)$
- $f(n) = 2^{\sqrt{log\ n}}, g(n) = \sqrt{n}$, $f(n) \in o(g(n))$ by comparing the logarithms
- $f(n) = 2^{\sqrt{log\ n}}, g(n) = 2^{(2^{\sqrt{log\ log\ n}})}$, $f(n) \in \omega(g(n))$ by comparing double logarithm
- $f(n) = n^{\frac{1}{log\ n}}, g(n) = n^{\frac{1}{log^2 n}}$, $f(n) \in \Theta(g(n))$ both are equal to 2
- $f(n) = n^{\frac{1}{log\ n}}, g(n) = n^{\frac{1}{\sqrt{log\ n}}}$, $f(n) \in o(g(n))$ since $f(n) = 1, g(n) \to \infty$

Solving Recurrences

Telescoping

- Goal: Make successive terms cancel out leaving first and last
- Mergesort: $T(n) = 2T(n/2) + n \Rightarrow \frac{T(n)}{n} = \frac{T(n/2)}{n/2} + 1$ and so on
- By expanding, we have $log\ n$ layers which cancel out to leave $\frac{T(n)}{n} = \frac{T(1)}{1} + log\ n$ and $T(n) = O(n\ log\ n)$
- $T(n) = aT(\frac{n}{b}) + f(n) \Rightarrow \frac{T(n)}{g(n)} = \frac{T(\frac{n}{b})}{g(\frac{n}{b})} + h(n), g(n) = ag(n/b), h(n) = \frac{f(n)}{g(n)}$

Substitution

- Guess form of solution, then verify by induction
- e.g. Tower of Hanoi: Solve $T(n) = 2T(n-1) + 1$, guess $T(n) = 2^n - 1$, then verify by induction
- Example: $T(n) = 4 \cdot T(n/2) + \sqrt{n}$
- Trying gives $T(n) \in O(n^3)$ and $T(n) \in \Omega(n)$, but nothing tight
- Guess $T(n) \in \Theta(n^2)$ using $T(n) \leq c_2 \cdot n^2 - c_1 \cdot n$ (to deal with the $\sqrt{n}$ term)
- For $c_1 = 1$, set c_2 large enough so $T(1) \leq c_2 - c_1$
- Upper Bound: $T(n) = 4 \cdot T(n/2) + \sqrt{n} \leq 4 \cdot (c_2 \cdot n^2/2^2 - c_1 \cdot n/2) + \sqrt{n} = c_2 \cdot n^2 - 2 \cdot c_1 \cdot n + \sqrt{n} \leq c_2 \cdot n^2 - c_1 \cdot n + (\sqrt{n} - c_1 \cdot n) \leq c_2 \cdot n^2 - c_1 \cdot n$
- Lower Bound: Guess $T(n) \geq c_3 \cdot n^2$ where $c_3 > 0$
- $T(n) = 4 \cdot T(n/2) + \sqrt{n} \geq 4 \cdot (c_3 \cdot n^2/2^2) + \sqrt{n} = c_3 \cdot n^2 + \sqrt{n} \geq c_3 \cdot n^2$
- Combining both gives $T(n) \in \Theta(n^2)$

Recursion Tree

- Draw recursion tree
- Multiply layers and sum of nodes in each layer

Master Theorem

- Solve recurrence of generic form $T(n) = aT(n/b) + f(n)$, $a \geq 1, b \geq 1, f \in \Omega(1)$
- Classify work into splitting/combining and solving base cases
- Solving base cases is linear in the number of leaves, n^d
- Tree Height: $log_b n$, Number of children of a node: a, Number of leaves: $a^{log_b n} = n^{log_b a} = n^d$, $d = log_b a$ is the critical exponent
- Cost of splitting and combining is $f(n)$
- Floors and ceilings within recursive subproblem do not affect asymptotic growth of the function

Cases

- 1: $f(n) \in O(n^{d-\epsilon}) \Rightarrow T(n) \in \Theta(n^d), \epsilon > 0$
- 2: $f(n) \in \Theta(n^d log^k n)$
  - 2a: $k \geq 0 \Rightarrow T(n) \in \Theta(n^d log^{k+1} n)$
  - 2b: $k = -1 \Rightarrow T(n) \in \Theta(n^d log\ log\ n)$
  - 2c: $k < -1 \Rightarrow T(n) \in \Theta(n^d)$
- 3: $f(n) \in \Omega(n^{d+\epsilon}) \Rightarrow T(n) \in \Theta(f(n)), \epsilon > 0$, requires regularity condition $af(n/b) \leq cf(n), c < 1$ to ensure $f(n)$ is dominant term (although actually redundant)

$$af(n/b) \leq cf(n)$$
$$\frac{a}{c}f(n/b) \leq f(n)$$
$$(\frac{a}{c})^i f(1) \leq f(b^i)$$
$$\Omega(n^{d+\epsilon}) \ni n^{log_b(\frac{a}{c})} f(1) = (\frac{a}{c})^{log_b n} f(1) \leq f(n)$$
$$\epsilon = log_b(\frac{a}{c}) - log_b a > 0, n = b_i$$

Glossary

- Dominant Term: Largest term in terms of time complexity
- Linearity: $f(x) \in \Theta(x)$, $f(x)$ is linear in x
- Polynomial: $f(n) = n^c$ for some $c \in \Theta(1)$, Polylog: $f(n) = (log\ n)^c$ for some $c \in \Theta(1)$

Proof of Correctness

Iterative Algorithm

- Goal: Prove a given algorithm is correct
- Loop Invariant (Induction Hypothesis): Desirable conditions that should be satisfied at the start of each iteration
- Initialisation (Base Case): Show invariants are true at the start of the first iteration
- Maintenance (Inductive Step): Show that if invariant is true at the start of current iteration then it is also true at the start of the next iteration
- Termination (Induction implies Correctness): If the loop invariant holds at the end of the last iteration, then the algorithm outputs a correct answer

Dijkstra's Proof

- Loop Invariant: At the start of an iteration, $\forall u \in R, d(u) = dist(s, u)$ (all processed vertices have the correct shortest distance)
- Observation 1: $d(v) = min_{u \in R}(d(u) + w(u, v)) \geq dist(s, v)$
- Termination: At end of computation, $R = V$, and all computed distances are correct
- Maintenance: Claim for selected v, $dist(s, v) = min_{u \in R}(d(u) + w(u, v))$
- Consider a shortest path P from s to v.
- Observation 2: All vertices in $P \backslash \{v\}$ must be in R
- If not, we should have taken x, the first vertex not in R along P, because $min_{u \in R}(d(u) + w(u, v)) \geq dist(s, v) = d(u') + w(u', v) \geq min_{u \in R}(d(u) + w(u, v))$
- Binary Heap: $O(log\ n)$ operations, overall $O((m+n)\ log\ n)$, Fibonacci Heap: $O(1)$ insert, $O(log\ n)$ extract, $O(m + n\ log\ n)$
- New Algorithm: $O(m\ log^{2/3} n)$, faster when $m \in o(n\ log^{1/3} n)$ (sparse graph)

Recursive Algorithms

- Base Case: Show correctness
- Inductive Step: Assume algorithm is correct for input size smaller than n, then show it is correct for size n

Binary Search Proof

- Base Case: Obvious
- Induct on array size, if mid is what we want, returns correctly, if not falls into a smaller and correct case
- Binary Search: Prove it works for one element, then induct on array size

Divide and Conquer

- Divide problem into smaller subproblems
- Solve subproblems recursively
- Combine subproblem solutions to get the solution of the the full problem
- Naive Matrix Multiplication: $\Theta(n^3)$
- Divide and Conquer: Split into quadrants, $T(n) = 8T(n/2) + \Theta(n^2), T(n) \in \Theta(n^3)$, no improvement
- Strassen's Algorithm: Split into quadrants, $T(n) = 7T(n/2) + 18 \cdot n^2, T(n) \in \Theta(n^{log_2 7}) \approx \Theta(n^{2.807\ldots})$
- H-Index: Find greatest i such that $A[i] \geq i$, this can be done using binary search in $O(n)$ time
- However, we can improve this to $O(log\ ans)$
- Try increasing 2^i where $i = 0, 1, 2, \ldots$ until we get $i^* \geq ans$
- Then $i^*/2 < ans \leq i^*$ so we can binary search on $[1, i^*]$ with cost $O(log\ i^*)$

Sorting

- Sort elements in an array in non-decreasing order
- Comparison Based: Elements can only be compared with each other and no other properties can be used
- Worst-case time complexity for comparison-based sorting is $\Omega(n\ log\ n)$
- Decision Tree: Every node asks a question, and takes a child depending on answer, at leaf an output is given
- Model a comparison based algorithm as a decision tree, which has $n!$ leaves minimum, and height at least $log(n!) \in \Theta(n\ log\ n)$ by Stirling's
- To merge k sorted arrays of size n, $\Theta(kn\ log\ k)$ is a tight lower bound using $T(k, n) = 2T(k/2, n) + ckn$
- The number of ways to combine is $\frac{(kn)!}{(n!)^k}$, so the complexity is $\Theta(kn\ log\ k)$ using $log(n!) \in n\ log\ n - n\ log\ e + O(log\ n)$
- Counting sort is not comparison-based, and finishes in $O(n + k)$ time
- Other Properties: Small running time (Worst / Average Case), Stability (Preserving original positions of equal items), In-Place (Uses $O(1)$ extra memory)
- Stooge Sort: Sort first $2n/3$, last $2n/3$, first $2n/3$, $T(n) = 3T(2n/3) + O(1)$, $T(n) \in O(n^{log_{3/2} 3}) = O(n^{2.7095\ldots})$

QuickSort Analysis

- $T(n) = T(j-1) + T(n-j) + cn$ if pivot is jth element
- Worst Case: $j = 1$ or $j = n$, $T(n) = T(n-1) + cn \in \Theta(n^2)$, proof by induction
- Average Case: $A(n)$ is expected running time when input is a uniformly random permutation π
- $A(n) = \sum_\pi \frac{1}{n!} \cdot$ (running time of quick sort on π)
- $A(n) = \frac{1}{n} \sum_{j=1}^{n} [A(j-1) + A(n-j) + cn]$
- $A(n) = cn + \frac{2}{n} \sum_{j=0}^{n-1} A(j)$
- $n \cdot A(n) = cn^2 + 2\sum_{j=0}^{n-1} A(j)$
- $(n-1) \cdot A(n-1) = c(n-1)^2 + 2\sum_{j=0}^{n-2} A(j)$
- $n \cdot A(n) - (n-1) \cdot A(n-1) = c(2n-1) + 2A(n-1)$
- $n \cdot A(n) - (n+1) \cdot A(n-1) = n \cdot A(n) - (n-1) \cdot A(n-1) - 2A(n-1) = c(2n-1)$
- $\frac{A(n)}{n+1} - \frac{A(n-1)}{n} = \frac{c(2n-1)}{n(n+1)} < \frac{c(2n+2)}{n(n+1)} = \frac{2c}{n}$
- $\frac{A(n)}{n+1} < 2c \cdot (\frac{1}{n} + \frac{1}{n-1} + \ldots + \frac{1}{2}) + \frac{A(1)}{2}$ by telescoping
- $A(n) \in O(n\ log\ n)$

Karatsuba's Algorithm

- Naive: To compute $C(x) = A(x) \times B(x)$ where they are polynomials, compute the coefficients of $C(x)$ in $O(n^2)$ with complete search
- Karatsuba's: Compute $A_1(x) \times B_1(x), A_2(x) \times B_2(x)$ still. However, instead of computing $A_1(x) \times B_2(x), A_2(x) \times B_1(x)$, instead compute $[A_1(x) + A_2(x)] \times [B_1(x) + B_2(x)]$. Then $A_1(x) \times B_2(x) + A_2(x) \times B_1(x) = [A_1(x) + A_2(x)] \times [B_1(x) + B_2(x)] - A_1(x) \times B_1(x) - A_2(x) \times B_2(x)$
- Then we have $T(n) = 3 \cdot T(n/2) + O(n)$ which is $T(n) \in \Theta(n^{log_2 3}) = \Theta(n^{1.58\ldots})$

Randomised Algorithms

- Utilise randomisation to develop algorithms that are more efficient or simpler than their deterministic counterparts at the cost of allowing a small error probability
- Verification of Matrix Multiplication: Check if square $AB = C$, takes $O(n^{2.807})$ by Strassen's
- Freivald's: Choose a size n column vector where each entry is random binary number with uniform probability, then check if $ABv = Cv$ in $O(n^2)$ time
- If $AB = C$, then $ABv = Cv$ will output correctly
- If not $AB \neq C$, consider $C^* = AB - C, u = C^* v$. It will output an incorrect answer iff all $u_k = 0$
- Principle of Deferred Decisions: Instead of fixing all random choices in advance, we conceptually defer the outcomes of random choices until the moment they are actually needed
- Once we reveal random numbers excluding v_j, the non v_j term is fixed, and there is at most one choice of it which makes $u_i = 0$
- The probability of success is $\geq 1/2$, and can be increased to $\geq 1 - 1/2^t$ by repeating t times

Coupon Collector

- Given n types of coupons, we want to obtain all, how many must we buy to collect all types of coupons
- Throw m balls in n bins, and check the probability that every bin contains at least one ball
- For a specific bin, the probability it has no balls is $(1 - 1/n)^m = (1 - 1/n)^{n \cdot m/n} \leq e^{-m/n}$ using $1 + x \leq e^x$
- The probability at least one bin is empty is at most $n(1 - 1/n)^m \leq ne^{-m/n}$
- The probability is at most $1/n$ if $m \geq 2n\lceil ln\ n \rceil$ since $e^{-2\ ln\ n} = \frac{1}{n^2}$
- So we can buy $m = 2n\lceil ln\ n \rceil \in \Theta(n\ log\ n)$ boxes for a success probability of at least $1 - \frac{1}{n}$
- Expected Value: $E[X] = \sum_x x \cdot Pr[X = x]$
- Markov Inequality: If X is a non-negative random variable and $a > 0$, then $Pr[X \geq a \cdot E[X]] \leq 1/a$
- e.g. If the expected runtime is at most t, the runtime is at most $100 \cdot t$ with probability at least 0.99, let $\alpha = 100$
- Linearity of Expectation: $E[A + B] = E[A] + [B]$
- Indicator Random Variable: Let ε be an event, then the indicator random variable 1_ε is defined as 1 if it occurs or 0 otherwise, so $E[1_\varepsilon] = Pr[\varepsilon]$

Quicksort Again

- Let $X_{i,j}$ be the number of comparisons made between a_i and a_j
- $E[comparisons] = E[\sum_{i<j} X_{i,j}] = \sum_{i<j} E[X_{i,j}]$ by LOE
- Let $\varepsilon_{i,j}$ denote the event $X_{i,j} = 1$
- We claim $Pr[\varepsilon_{i,j}] = \frac{2}{j-i+1}$ for $i < j$
- A comparison is made between them only if the pivot chosen is either one of them, and they must start in the same array
- Each number is equally likely to be the pivot, and from $(a_i, \ldots, a_j)$, there are $j - i + 1$ elements
- $E[comparisons] = \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{1}{j-i+1} = 2\sum_{i=1}^{n-1} (\frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{n-i+1}) \in O(n\ log\ n)$
- Randomised quick sort finishes in $O(n\ log\ n)$ time with probability at least 0.99 by Markov Inequality

Summary of Techniques

- Las Vegas: Always correct, but time complexity guarantee is only in expectation
- Monte Carlo: Output is correct with some probability, but the time complexity guarantee always holds
- Principle of Deferred Decision: If we can show $\forall x, Pr[\varepsilon | X = x] \geq p$, then $Pr[\varepsilon] \geq p$
- Success Probability Amplification: The error probability can be reduced to at most f by repeating the algorithm $\lceil log \frac{1}{f} \rceil$ times
- Union Bound: Upper bound the probability that bad ε occurs, and you know $Pr[\varepsilon] = Pr[\varepsilon_1 \vee \ldots \vee \varepsilon_n] \leq Pr[\varepsilon_1] + \ldots + Pr[\varepsilon_n]$
- Then to make sure $Pr[\varepsilon] \leq f$, we just need to make sure $Pr[\varepsilon_i] \leq \frac{f}{n}$ for each i
- Markov Inequality: If X is a non-negative random variable and $a > 0$, then $Pr[X \geq a \cdot E[X]] \leq \frac{1}{a}$
- Can be used to turn Las Vegas algorithm into Monte Carlo algorithm via Markov inequality
- Set $Pr[\text{runtime} \geq 100 \cdot T(n)] \leq Pr[\text{runtime} \geq 100 \cdot \text{expected runtime}] \leq \frac{1}{100}$, then the runtime is $\in \Theta(T(n))$ with $Pr \geq 0.99$, just set a time limit of $100 \cdot T(n)$
- Linearity of Expectation: $E[A + B] = E[A] + E[B]$

Transmission

- Goal: Check whether two n-bit strings are equal
- Method: Create a set S of n^2 smallest primes
- Sample a random prime p from S
- Send p and $S_A\ mod\ p$
- Check $S_B\ mod\ p$ against $S_A\ mod\ p$
- When they are equal, the output will be correct
- When they are not equal, the output is incorrect iff p divides $|S_A - S_B|$, let this value be d
- Then $1 \leq d < 2^n$
- d has at most $n - 1$ distinct prime factors, so the probability p is a divisor of d is at most $\frac{n-1}{|S|} = \frac{n-1}{n^2} = \frac{n}{n^2} - \frac{1}{n^2} = \frac{1}{n} - \frac{1}{n^2} \leq \frac{1}{n}$
- Any correct algorithm requires $\Omega(n)$ bits by decision trees

Partition of Graph

- Partition a graph randomly by assigning each vertex in the set V_1 or V_2 to get the expected number of edges crossing to be $\frac{|E|}{2}$
- To improve slightly, pick one specific edge and force it to cross, then we get $\frac{|E|+1}{2}$ which is marginally better
- If $|V|$ is even, we can select V_1 uniformly at random from all size $|V|/2$ subsets of V and set V_2 as the complement
- $Pr(u \in V_1) = \frac{1}{2}$
- $Pr(v \in V_2 | u \in V_1) = 1 - Pr(v \in V_1 | u \in V_1) = 1 - \frac{|V|/2-1}{|V|-1} = \frac{|V|/2}{|V|-1}$

- We can sum the symmetric case to get $E[X_e] = 2 \cdot Pr(u \in V_1, v \in V_2) = 2 \cdot Pr(u \in V_1) \cdot Pr(v \in V_2 | u \in V_1) = 2 \cdot \frac{1}{2} \cdot \frac{|V|/2}{|V|-1} = \frac{|V|/2}{|V|-1}$
- By linearity of expectation, $E[X] = \sum_{e \in E} E[X_e] = |E| \times \frac{|V|/2}{|V|-1} = \frac{|E|}{2} \times \frac{|V|}{|V|-1}$

Reservoir Sampling

- Given a singly linked list, select a random node
- Without knowing the size of the linked list beforehand, hard to ensure each node is chosen with the same probability
- Reservoir sampling allows us to keep the uniform probability

```
i = 0
while cur != None:
    i += 1
    v = U(0, 1)
    if v <= 1.0 / i:
        ans = cur
    cur = cur->next
return ans
```

Dynamic Programming

- Divide problem into smaller overlapping subproblems
- Solve the subproblems recursively with memoisation
- Combine the subproblem solutions to get the solution of the full problem
- Bottom Up: Compute the values in the array in some order, starting from base cases

LCS

- C is a subsequence of A if C can be obtained by removing elements from A
- LCS: Given two arrays, output a longest common subsequence C, or equivalently, find a largest non-crossing matching
- Brute Force: Check all possible subsequences of A and see if they fit in B, takes $O(m \cdot 2^n)$ time
- Recursive Approach: Base cases: $i = 0$ or $j = 0$
- Inductive Step: Compute $LCS(n, m)$ given $LCS(n-1, m), LCS(n, m-1), LCS(n-1, m-1)$
- If $a_n \neq b_m$, take the longer of $LCS(n-1, m), LCS(n, m-1)$
- If $a_n = b_m = x$, then take $LCS(n-1, m-1) \circ x$
- Use cut and paste to prove observation for equality case
- Show $LCS(n, m)$ should terminate with a_n when $a_n = b_m$
- Suppose not and it ends with something else
- We can then append a_n and get a longer subsequence, which is a contradiction
- Optimal Substructure: An optimal solution to a problem can be constructed from optimal solutions to its subproblems
- With memoisation, takes $O(mn)$ time

Knapsack

- Pack a set of items into a container with a capacity constraint to maximise the total value
- Formally, output a subset $S \subseteq [n]$ such that $\sum_{i \in S} w_i \leq W$ maximises $\sum_{i \in S} v_i$
- Suppose S is an optimal solution
- If $n \in S$, then $S \backslash \{n\}$ is an optimal solution to the subproblem of n with $W - w_n$
- If $n \notin S$, then S is an optimal solution to the subproblem without n
- If not, a better solution can be obtained by cut and paste, by replacing S with an optimal solution to the subproblem
- In the first case, we have $m[n, W] = m[n-1, W-w_n] + v_n$, and in the second case, we have $m[n, W] = m[n-1, W]$, giving $O(nW)$

Change-Making Problem

- Find the minimum number of coins of specific k denominations that add up to n cents
- $M[n] = 1 + min_{i \in [k]} M[n - d_i]$, $O(nk)$ time
- Can use cut and paste on the optimal solution for $n - d_i$

Cut and Paste

- Assume there is an optimal solution to S to some problem instance
- Look at the sub-solution to S that corresponds to some smaller problem by cutting out some part of the optimal solution
- If the sub-solution were not the optimal one for the subproblem, we could cut it out, and paste in the truly optimal solution for the sub problem
- This produces a new solution which has strictly greater value
- This contradicts the assumption that S was optimal unless the sub-solution was also optimal
- If an optimal solution did not use a specific choice, we can use cut and paste to build a different optimal solution that does

Bellman Ford

- If a path $P(s, v)$ is the shortest path, then any subparth is also a shortest path
- Let $L(v, i)$ be the weight of the shortest path $P(s, v)$ with $\leq i$ edges
- We want to compute $L(v, n-1)$ as the longest simple path with $n-1$ edges
- Either $L(v, i) = L(v, i-1)$ or $L(v, i) = min_{(x,v) \in E} L(x, i-1) + \omega(x, v)$, representing the optimal substructure property
- We iterate $n-1$ times, where for each vertex, we relax each adjacent edge
- WE can simplify this by just relaxing each edge without the by vertex part to get $O(|V||E|)$ time and $O(|V|)$ space

Greedy Algorithms

- Recast the problem so that only one subproblem needs to be solved at each step
- Often beats other methods but is often not applicable

Fractional Knapsack

- Knapsack but we can take fractions of items
- Optimal Substructure: If we remove y kg of item j from the optimal knapsack, then the remaining load must be optimal for weight at most $W - y$ kg from the remaining $n-1$ items and $w_j - y$ kg of item j
- Cut and Paste: Let X be the value of the optimal knapsack, and suppose that the remaining load after removing y of item j was not the optimal knapsack for $W - y$ from $n-1$ items and $w_j - y$ of item j
- Then there is a knapsack with value $> X - v_j \cdot \frac{y}{w_j}$ with weight $\leq W - y$ among $n-1$ other items and $w_j - y$ of item j
- Combining y of j gives knapsack with value $> X$ and weight $\leq W$ which is a contradiction
- So there must be optimal substructure
- Greedy Choice Property: Let j^* be the item with maximum v_j / w_j. Then there exists an optimal knapsack containing $min(w_{j^*}, W)$ of item j^*
- Suppose an optimal knapsack has $x_{j^*} < min(w_{j^*}, W)$. From the rest of the knapsack, pick y such that $\sum y = min(w_{j^*}, W) - x_{j^*}$. Then we can replace these by $min(w_{j^*}, W) - x_{j^*}$ of item j^*, and total value does not decrease since j^* has maximum ratio
- Greedy Algorithm: Use the greedy choice property to take as much j^* as possible, then repeat until we run out of weight, $O(n \log n)$

Greedy Algorithm Paradigm

- Cast the problem where we have to make a choice and are left with one subproblem to solve
- Prove (using exchange argument) that there is always an optimal solution to the original problem that makes the greedy choice so the greedy choice is safe
- Use optimal substructure (proof by contradiction or cut-and-paste argument) to show that we can combine an optimal solution to the subproblem with the greedy choice to get an optimal solution to the original problem

Huffman Code

- Binary Coding: Mapping from alphabet set to binary string
- Fixed Length Coding: Each alphabet is a binary string of fixed length, $m\lceil log_2\ n \rceil$ bits needed for m alphabets with alphabet size n
- Letter Frequency Variation: Give more frequent alphabets encoding with shorter bit strings
- Average Bit Length: $ABL(\gamma) = \sum_{x \in A} f(x) \cdot |\gamma(x)|$ where f is the frequency and γ is the encoding
- Need to be careful of ambiguity caused when the encoding of one letter is a prefix of another
- Prefix Coding: A coding where the above ambiguity does not exist
- Problem: Find a prefix coding γ where $ABL(\gamma)$ is the minimum
- Huffman Coding: Labeled binary tree where each child is either a 0 or 1 with a frequency, and leaf nodes represent alphabets
- For each prefix code of a set of alphabets, there exists a binary tree where there is a bijective mapping between the alphabets and leaves, and the label of a path from root to leaf corresponds to the prefix code of the corresponding alphabet at the leaf
- $ABL(\gamma) = \sum_{x \in A} f(x) \cdot depth_T(x)$
- Observation 1: The ideal binary tree should be a full binary tree where every internal node has degree 2, if not we have redundant nodes
- More frequent alphabets should be closer to the root
- Let $a_1, a_2, ..., a_n$ be the alphabet in non-decreasing order of frequency (a_1 is least frequent)
- Observation 2: The least frequent alphabet a_1 cannot be present at a higher level than the deepest one, since swapping it with a lower one cannot increase ABL
- Observation 3: For a_2, it can be the sibling of a_1
- Lemma: There exists an optimal prefix coding in which a_1, a_2 appear as siblings (not necessarily in all!)
- Observation 4: For siblings a_1 and a_2, we can merge them into a new $f(a')$
- $OPT_{ABL}(A)$: Minimum ABL over all prefix codes for alphabet A, $OPT(A)$: Prefix code for alphabet A with $ABL = OPT_{ABL}(A)$
- Let $A' = a_3, ...a', ...a_n$ be $n-1$ alphabets with $f(a') = f(a_1) + f(a_2)$
- We claim that $OPT_{ABL}(A) = OPT_{ABL}(A') + f(a_1) + f(a_2)$, and this implies the algorithm is optimal
- Algorithm
  - If $|A| = 2$, return a single node with a_1, a_2 as children
  - If not, repeatedly remove a_1, a_2 from A
  - Create letter a' and calculate $f(a') = f(a_1) + f(a_2)$
  - Insert a' into A
  - Update tree to $OPT(A)$
  - Replace leaf node a' in tree with node containing children a_1, a_2
  - Heap for alphabet and frequencies is $O(log\ n)$ insertion and removal, overall $O(n\ log\ n)$
- Now we need to prove $OPT_{ABL}(A) = OPT_{ABL}(A') + f(a_1) + f(a_2)$
- Let T' be the optimal binary tree for $OPT_{ABL}(A')$, then construct T by replacing a'
- This gives $ABL = OPT_{ABL}(A') + f(a_1) + f(a_2)$, with proof below
  - Let d be the depth of a' in T' with ABL $OPT_{ABL}(A')$
  - For the tree we constructed for A, we have $ABL =$

$$= \sum_{i=1}^{n} f(a_i) \cdot depth_T(a_i)$$
$$= \sum_{i=3}^{n} f(a_i) \cdot depth_T(a_i) + f(a_1) \cdot (d+1) + f(a_2) \cdot (d+1)$$
$$= \sum_{i=3}^{n} f(a_i) \cdot depth_T(a_i) + (f(a_1) + f(a_2)) \cdot d + f(a_1) + f(a_2)$$
$$= \sum_{i=3}^{n} f(a_i) \cdot depth_T(a_i) + f(a') \cdot depth_{T'}(a') + f(a_1) + f(a_2)$$
$$= \sum_{i=3}^{n} f(a_i) \cdot depth_{T'}(a_i) + f(a') \cdot depth_{T'}(a') + f(a_1) + f(a_2)$$
$$= OPT_{ABL}(A') + f(a_1) + f(a_2)$$

- For T' corresponding to $OPT_{ABL}(A')$, splitting a' gives a coding with $ABL = OPT_{ABL}(A') + f(a_1) + f(a_2)$, and thus $OPT_{ABL}(A) \leq OPT_{ABL}(A') + f(a_1) + f(a_2)$ by construction
- For T corresponding to $OPT_{ABL}(A)$, replacing a_1, a_2 with a' gives $ABL = OPT_{ABL}(A) - f(a_1) - f(a_2)$, and implies $OPT_{ABL}(A') \leq OPT_{ABL}(A) - f(a_1) - f(a_2)$ by construction as well
- Combining them gives the recurrence we want by showing equality

Interval Scheduling

- Given a set of intervals, find the largest non-overlapping set
- Choose the interval that starts last, and recurse (alternatively, earliest end time)
- Optimal Substructure: Suppose an optimal scheduling contains an interval a_i, then split the remaining intervals for the set before and after this activity, then the optimal scheduling must contain the optimal scheduling for both new subsets created
- Greedy Choice Property: Greedily select the interval that starts last. Consider an optimal scheduling S where a is the interval that starts last in S. We can always replace a with a^* which is the interval that actually starts last. The new schedule is compatible with the same number of activities, so it is also optimal
- Greedy Algorithm: Sort by start time in non-decreasing order, then iterate from the back and select the latest starting activity that is compatible

MST

- Given an MST T of a graph, if we remove any edge in T, it is partitioned into two substrees T_1 and T_2
- Optimal Substructure: We know $w(T) = w(T_1) + w(u, v) + w(T_2)$, so if there was T_1' with lower weight than T_1, then the union T' will cause a contradiction
- Greedy Choice: The least weight edge connecting A to $V \backslash A$ must be in T
- Suppose there is an MST T without this edge, and consider the unique simple path between them in T
- If we swap it out with the first edge on the path that connects a vertex in A to a vertex in $V \backslash A$, we get a lighter weight spanning tree which is a contradiction

Minimum Rectangle Cover

- Optimal Substructure: Start with an optimal solution, then remove some rectangle. If we remove it, we are left with a subproblem where we have to cover the rest of the points. If this subproblem is not optimal, then we can use the more optimal solution and add the removed rectangle back to get a more optimal solution, which is a contradiction
- Greedy Choice: Cover the leftmost point with one rectangle left aligned to it. Suppose there is an optimal solution that covers the leftmost point differently, we can always slide this rectangle so that its leftmost side is aligned to the leftmost point like our greedily constructed rectangle and we still cover all points, so the greedy choice is safe
- Solution: Sort points by non-increasing x-coordinate, then one pass and use the greedy strategy

Minimum Operations to Halve Array Sum

- Optimal Substructure: We start with an optimal solution that halves the sum. After choosing some number N, it reduces $X - \frac{N}{2}$ to $\frac{X}{2}$. This subproblem must be optimal, if not we can take the more optimal solution and add back $\frac{N}{2}$ to have a more optimal solution, a contradiction
- Greedy Choice: Halve the largest element L. Suppose there is an optimal solution which halves some other N, since $L \geq N$, halving L will not be worse off and the greedy choice is safe
- Use a heap to keep track of the current number, and run it until the total sum is halved

Amortised Analysis

- $T(n) = \sum_{i=1}^{n} t(i) = nf(n)$ could be wrong if not all operations have the same cost
- Amortised analysis shows that the average cost per operation can still be small even if a few operations might be expensive
- Amortised analysis guarantees the average performance of each operation in the **worst case**

Aggregate Method

- Binary Counter: Find total number of bit flips for n increments of a k-bit binary counter
- Let $t(i)$ be the number of bit flips in the i-th operation, then we have $T(n) = \sum_{i=1}^{n} t(i)$
- In the worst case, $t(i) = k, T(n) = n \cdot k \in O(n \cdot k)$, but this is not a tight bound
- Let $f(j)$ be the number of times the j-th bit flips, then $T(n) = \sum_{j=0}^{k-1} f(j)$. $f(0) = n, f(1) = n/2, ..., f(j) = n/2^j$. This gives $T(n) = n \cdot (1 + 1/2 + ... + 1/2^{k-1}), T(n) < n \cdot 2$
- Then $k \approx log\ n$
- The cost per increment is then $T(n)/n < 2 \in O(1)$
- Lacks precision of the other methods which allow a specific amortised cost to be allocated to each operation

Accounting Method

- Charge ith operation an amortised cost $c(i)$ with $\$1$ representing 1 unit of time or work
- This is consumed to perform the operation and any amount not used can be stored for the future
- Impose an extra charge on inexpensive but frequent operations to pay for future expensive but rare operations
- Balance must never be negative
- Ensure $\sum_{i=1}^{n} t(i) \leq \sum_{i=1}^{n} c(i)$ to provide an upper bound
- Binary Counter: Charge $\$2$ for $0 \to 1$, with $\$1$ going to this and the other $\$1$ paying for a future $1 \to 0$
- Every 1 bit has $\$1$ in the bank at any point
- These savings are used to pay for resetting $1 \to 0$
- After i increments, the amount of money in the bank is the number of 1s in the binary representation of i, which cannot be negative
- The amortised cost is thus $\$2 \in O(1)$ for each increment

Potential Method

- ϕ: Potential function, $\phi(i)$: Potential after ith operation
- Important Conditions: $\phi(0) = 0, \forall i, \phi(i) \geq 0$
- Amortised cost of ith operation $=$ Actual cost of ith operation + $\Delta\phi_i$ or $\phi(i) - \phi(i-1)$
- Amortised cost of n operations $= \sum_i$ Amortised cost of ith operation $=$ Actual cost of n operations $+\phi(n) \geq$ Actual cost of n operations, by telescoping
- Upper bound actual cost using amortised cost
- Select a suitable ϕ so that the costly operation $\Delta\phi_i$ is negative to the point where it nullifies the effect of actual cost, try to find a quantity that is decreasing during the expensive operation
- Binary Counter: Since number of 1s decreases at the expensive operation, set this to be the potential
- Actual Cost of ith increment $= 1+$ Length of 1 suffix
- Actual Cost of ith increment $= l_i + 1, \Delta\phi_i = -l_i + 1$ since $\phi(i) = x - l_i + 1, \phi(i-1) = x$
- Amortised Cost of ith increment $= l_i + 1 - l_i + 1 = 2 \in O(1)$

Dynamic Table Insertion

- Resizable table which grows by reallocation e.g. C++ Vector
- Naive: Grow table size by one each insertion, $O(n^2)$ for n insertions
- Doubling: Double table size when full, worst-case is $O(n)$ for one insertion
- Heavy operation only occurs when $n-1$ is a power of 2, otherwise it is $O(1)$
- Aggregate Method: Let $t(i)$ be the cost of the ith insertion, it can be split between $t_1(i) = i$ if $i-1$ is an exact power of 2, and $t_2(i) = 1$ for all i
- By the aggregate method,

$$T(n) = \sum_{i=1}^{n} (t_1(i) + t_2(i))$$
$$T(n) = \sum_{i=1}^{n} t_1(i) + \sum_{i=1}^{n} t_2(i)$$
$$T(n) = \sum_{j=0}^{log(n-1)} 2^j + \sum_{i=1}^{n} 1$$
$$T(n) = 2^{log(n-1)+1} - 1 + n$$
$$T(n) \leq 2 \cdot (n-1) + n$$
$$T(n) \leq 3 \cdot n$$

- Thus the average cost is $3 \in O(1)$
- Potential Method: Since table is always at least half-full, consider $\phi(i) = 2i - size(T) \geq 0$, since $size(T) \geq 0$
- When table is not full, actual cost $= 1$, $\Delta\phi_i = 2$, amortised cost $= 3$ since $\phi(i-1) = 2(i-1) - l, \phi(i) = 2i - l$
- When table is full, actual cost $= i$, $\Delta\phi_i = 3 - i$, amortised cost $= 3$ since $\phi(i-1) = i - 1, \phi(i) = 2$
- Accounting: Charge $\$3$ per insert: 1 for actual insert, and 2 for future copies, which pays for future resizing

WA2 Counters

- Increment, if $C[i] \mod 3 = 0$, invoke Increment($i+1$), Increment($i+2$)
- We want to show the amortised cost is $O(1)$
- Assign a cost of 3, and show the bank balance never goes negative
- Show the balance is equal to $2\sum_i (C[i] \mod 3)$, base case holds
- For induction, perform an increment and consume 1 from the budget
- If there is no recursive call, the bank balance increases by 2
- If there are recursive calls, then the bank balance decreases by 4, and we have a total of $2 + 4 = 6$ available, so we can allocated 3 to each recursive call

Reductions and Intractability

Reduction

- Example: The problem of longest palindromic subsequence of a string reduces to finding the LCS of a string and its reverse
- Convert an instance α of A to an instance β of B, solve β, and use solution of β to obtain solution of α, then A reduces to B
- Suppose we have MAT-MULTI ($A \cdot B$) and MAT-SQRT (C^2)

- Then MAT-SQR reduces to MAT-MULTI using $A = B = C$
- Then MAT-MULTI reduces to MAT-SQR using $C = \begin{bmatrix} 0 & A \\ B & 0 \end{bmatrix}, C^2 = \begin{bmatrix} AB & 0 \\ 0 & BA \end{bmatrix}$
- $p(n)$-time Reduction: A can be reduced to B, and the conversion steps take at most $p(n)$ time
- Running Time Composition: If there is a $p(n)$-time reduction from A to B and a $T(n)$-time algorithm to solve B, then there is a $T(p(n)) + O(p(n))$ time algorithm to solve A
- Polynomial Time Reduction: If there is an $O(n^c)$ time reduction from A to B for some constant c, then there is a polynomial time reduction from A to B or $A \leq_p B$
- If there is a polynomial time algorithm for B, there is also one for A, contraposition is also true
- If B is easy, then so is A; if A is hard, then so is B
- Polynomial functions are closed under composition: If $T(n), p(n)$ are polynomial, then $T(p(n))$ is also polynomial
- Polynomial time usually considers low c, and is a robust measure which is not reliant only computing models or hardware
- When an algorithm takes polynomial time, we usually mean that the runtime is polynomial in the length of the encoding of the problem instance (number of bits, usually most natural encoding)
- Psuedo-Polynomial Algorithm: Runs in time polynomial in numeric value of input (could be exponential in the length of the input)
- Iterative Fibonacci is not a polynomial-time algorithm since it is exponential in terms of input bit length, but it is pseudopolynomial
- Knapsack ($O(nW)$) is not while Fractional Knapsack ($O(n \log n)$) is since input needs $O(n \log W)$ bits

Intractability

- Computational Complexity Theory: Studying sets of computational problems and not individual computational problems
- Decision problem: Maps instance space to Yes/No
- Optimisation Problem: Outputs a optimised value instead
- We can convert an optimisation problem into a decision problem, so the decision problem is no harder than the optimisation problem
- Similarly, we can use binary search on the optimisation problem to solve the decision problem
- The decision problem can be solved in polynomial time iff the optimisation problem can be solved in polynomial time
- Karp Reduction: Given decision problems A, B, a polynomial time reduction from A to B, $A \leq_p B$, is a transformation from α to β such that α is a YES for A iff β is a YES for β and the transformation is polynomial time in the size of α
- If A is "hard", so is B, if B is "easy", so is A
- Example: Vertex Cover (Subset of vertices where every edge is covered) $\leq_p$ Independent Set (Subset of vertices where no edge has both endpoints inside)
- $X \subseteq V$ is a vertex cover iff $V \backslash X$ is an independent set
- The transformation $(G, k) \to (G, |V| - k)$ from VC to IS can be done in polynomial time, so we just need to show (G, k) is a YES iff $(G, |V| - k)$ is also a YES
- It can also be proven that Independent Set $\leq_p$ Vertex Cover

NP Class

- A verifier for a decision problem X is an algorithm A that takes in an instance I of X and a certificate / proposed solution S and outputs YES/NO, verifying whether the certificate is right or wrong
- I is a YES instance of X iff there exists a string s such that A outputs YES on inputs (I, s)
- s must be short and polynomial in the size of I to keep verification efficient, A must run in polynomial time
- NP: The set of all decision problems with polynomial time verifiers, "Non-deterministic polynomial time"
- Hamiltonian Cycle: Is there a cycle that visits each vertex exactly once
- Certificate is the cycle itself, while the verifier checks the conditions in polynomial time, so HC is in NP

P Class

- P: Polynomial time, the set of all decision problems which have an efficient polynomial-time algorithm
- $P \subseteq NP$ since we can just run the algorithm to verify

NP Hard and NP Complete

- NP Hard: $\forall A \in NP, A \leq_p X$, X is at least as hard as any other problem in NP
- NP Complete: X is in NP and is NP Hard
- Circuit Satisfiability: Given a DAG where vertices are logic gates and leaves are binary inputs, is there any binary input which outputs 1?
- The certificate is the binary input, so C-SAT is in NP
- For any problem in NP, consider the efficient verifier Q, which can be represented as a C-SAT DAG with output being the decision, so $\forall A \in NP, A \leq_p C - SAT$
- Literal: Boolean variable or negation e.g. $x_i, \overline{x_i}$
- Clause: Disjunction (OR) of literals e.g. $C_j = x_i \vee \overline{x_2} \vee x_3 \vee x_4$
- Conjunctive Normal Form: Conjunction (AND) of clauses e.g. $\Phi = C_1 \wedge C_2 \wedge C_3$
- CNF-SAT: Given a CNF formula, does it have a satisfying truth assignment?
- 3-SAT: CNF-SAT where each clause has exactly 3 literals for different variables, e.g. $\Phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$
- $C - SAT \leq_p CNF - SAT \leq_p 3 - SAT$ so 3-SAT is NP-Complete
- NP-Complete is a subset of NP which does not intersect P, NP-Hard intersects NP but not NP and contains NP-Complete
- If any NP-complete problem can be solved in polynomial time, then $P = NP$

Showing NP Completeness

- First show that $X \in NP$
- Then pick a known problem A which is NP-Complete and show $A \leq_p X$ so that X is NP-Hard
- Independent Set: No adjacent vertices, check if size $\geq k$
- We can show $IS \in NP$ by using the independent set as a certificate and verifying / checking the size of it in polynomial time
- Now we show $3 - SAT \leq_p IS$
- Make each clause a clique, and connect the complemented literals together
- This gives us an IS instance with k equal to the number of clauses
- Similarly we can show the other direction, so IS is NP-Complete
- Since we know $IS \leq_p VC$, VC is also NP complete
- Clique (find a clique with minimum size) is NP complete
- Hitting Set: Given a set of sets S, H is hitting if $H \cap S_i \neq \emptyset$ for all S_i
- The hitting set of size at most k can act as a certificate and can be verified in polynomial time
- $VC \leq_p HS$: Take an instance of vertex cover, then for each edge, make a set containing its endpoints
- The transformation can be done in polynomial time, and the hitting set is $\{S_e\}$
- G has a vertex cover of size at most k iff the other set has a hitting set of size at most k

NP-Complete Problem List

- SAT, 3SAT: Given a Boolean formula (usually in CNF), is there an assignment of variables such that it evaluates to true? 3SAT is the special case where each clause has exactly 3 literals
- Clique: Does a graph contain a clique of size $\geq k$?
- IS, VC: Does a graph contain a set of $\geq k$ vertices which are not adjacent? Does a graph contain a subset $C \subseteq V$ of size $\leq k$ such that every edge in E is covered by at least one vertex in C?
- 3-Colourable: Can the vertices of a graph be coloured with ≤ 3 colours so that no adjacent vertices have the same colour?
- Hamiltonian Cycle / Path: Does a graph have a simple cycle / path that visits every vertex exactly once?
- TSP: Given a complete weighted graph, is there a Hamiltonian Cycle with total cost $\leq K$?
- Subset Sum, Partition: Given a set of integers, is there a subset with sum T? Is there a partition into two subsets of equal sum?
- 3 Dimensional Matching: Given three sets and a subset of triples, is there a matching such that each element appears in exactly one triple? (Matching: Subset of cartesian product)
- Set Cover: Given a collection of subsets, is there a selection of $\leq k$ of them whose union equals the universe?
- 0-1 Knapsack: Given items with values and weights as well as a capacity and target value, is there a subset of items with total weight $\leq W$ and total value $\geq V$?

NP-Completeness of Subset Sum

- We can simply verify the certificate, so it is in NP
- We will show $3 - SAT \leq_p SUBSET - SUM$
- Given n variables x_i and m clauses c_j
- For each x_i, construct t_i, f_i of $n + m$ digits
- The digit position corresponding to this literal is 1, and for the clauses, it is 1 in t_i if x_i is in the clause, and 1 in f_i if $\overline{x_i}$ is in the clause, the rest of the digits are 0
- Construct x_j, y_j, where for each clause there are exactly 2 where the corresponding position is a 1, these are the slack numbers which we can use to top up
- Now for the subset sum, we set each of the literals to be 1, and each of the clauses to be 3
- Each literal or its complement can only be taken once
- Each clause can only use at most 2 slack numbers, so it must have at least one satisfied literal

WA2 Reductions

- Extend-MIS-in: Given $S \subseteq V$, is there an MIS I of G such that $S \subseteq I$
- Extend-MIS-out: Given $S \subseteq V$, is there an MIS I of G such that $I \cap S = \emptyset$
- Extend-MIS: Given $S_i \subseteq V, S_o \subseteq V, S_i \cap S_o = \emptyset$, is there an MIS I of G such that $S_i \subseteq I, I \cap S_o = \emptyset$
- Extend-MIS-unique: Given $S_i \subseteq V, S_o \subseteq V, S_i \cap S_o = \emptyset$, is there exactly one MIS I of G such that $S_i \subseteq I, I \cap S_o = \emptyset$
- Extend-MIS-in can be solved in polynomial time by verifying S's independence
- Extend-MIS $\leq_p$ Extend-MIS-out, by adding a new vertex v' adjacent to each $v \in S_i$, then let S' be the union of S_o and these v'

WA3 Reductions

- Define U-3-SAT to be a variant of 3-SAT where for each clause the number of literals that evaluate to TRUE is in U
- U-3-SAT is in NP since we can count the number of literals and evaluate
- $\{0\}$-3-SAT is in P
- $\{0, 3\}$-3-SAT is in P, construct a graphs where we add an edge between 2 literals in the same clause depending on their signs, then we can check that it is bipartite
- $\{0, 1, 2\}$-3-SAT is NP-Complete since we can flip the truth assignment of every variable from $\{1, 2, 3\}$-3-SAT
- min-MIS: Does G have an MIS $S \subseteq V$ with $|S| \leq k$
- min-VC: Does G have a vertex cover $C \subseteq V$ with $|C| \leq k$

- Extend-MIS-out is in NP since we can simply check for independence and maximality
- 3-SAT $\leq_p$ Extend-MIS-out so it is NP Complete
- Create a vertex for each literal, then for each variable, add a vertex (to S) and make a clique with the clause vertexes, so that either the literal or its complement is taken
- For each clause add a vertex adjacent to each literal and add this clause vertex to S so that at least one literal must be in the MIS
- min-MIS is in NP since we can easily verify the certificate
- min-VC $\leq_p$ min-MIS, so min-MIS is NP Complete
- Let $G = (V, E), k' = |V| + k$, the construct a new graph G'
- For each edge e, replace e with $k' + 1$ internally vertex-disjoint paths of length 2 between u, v
- For each vertex v, add a new path (v, v', v'')
- Suppose $C \subseteq V$ is a VC with $|C| \leq k$, construct an MIS I of G' as follows
- For $v \in C$, add v, v'' to I
- For $v \in V \backslash C$, add v' to I
- By construction, I is independent, and is maximal since C was a VC, then $|I| = |V| + |C| \leq k'$
- Suppose I is an MIS of G' with $|I| \leq k' = |V| + k$, construct a VC C of G as follows
- Each original edge was replaced by $k' + 1$ paths of length 2, so if any of the middle vertices belong to I, then all of them do, then $|I| > k'$ which is a contradiction
- So none of them can be in I, and every vertex must come from (v, v', v'')
- If only v' is in I, then do not include it
- If v, v'' are in I, then include v
- From the construction, $|C| = |I| - |V| \leq k$, and the maximality of I implies that for every original edge at least one must be in I so the corresponding vertex is added to C

Linear Time Sorting

- Counting Sort (Unstable Version): Create an array with size corresponding to range, then iterate and output the correct number of each, complexity is $\Theta(n + k)$
- Tradeoff is memory, especially if range of k is large
- Can be made stable, use prefix sums to find index where element belongs, iterate from back

```
C = [0]*(k+1) # frequency lookup
for i in A:
    C[i] += 1
for i in range(1, k+1):
    C[i] += C[i-1] # compute prefix sum
B = [0]*n # output array
for i in range(n-1, -1, -1): # iterate in reverse
    C[A[i]] -= 1 # decrease first
    B[C[A[i]]] = A[i] # place A[i] at index C[A[i]]
print(*B)
```

- Radix Sort: Iterated Counting Sort, Sort from LSD to MSD using **stable** counting sort, correctness can be proved by induction
- With d digits, Radix Sort runs in $O(d \cdot (n + k))$ time
- Setting $k = 10$ might not be the best setup
- Break a b-bit word into $\frac{b}{r}$ groups of r-bit words, so that there are $d = \frac{b}{r}$ passes, taking $O(\frac{b}{r}(n + 2^r))$ time
- Choose $r \approx log_2 n$, giving total time of $O(\frac{b}{log_2 n} \cdot (n + 2^{log_2 n})) = O(\frac{b \cdot n}{log_2 n})$
- If $b = log_2 n^d$, then Radix sort runs in $O(d \cdot n)$ time

Order Statistics

- Given unsorted array, find index of ith smallest element
- Naive: Repeatedly find minimum and remove, $O(n^2)$ worst case
- Sorting: Sort and return correct index, $\Theta(n\ log\ n)$
- Lower Bound: Selection takes $\geq n$ steps, since we must at least consider each element before making a conclusion

Quickselect

- Requires partition algorithm which runs in $\Theta(n)$
- Randomly choose pivot, partition, then recurs on corresponding half
- Partition returns final correct rank k of randomly chosen pivot
- If $k = i$, return pivot
- If $k > i$, recurse on $[l, k-1]$
- If $k < i$, recurse on $[k+1, r]$
- $O(n)$ best case, $O(n^2)$ worst case without randomness
- Let X_k be the indicator random variable which is 1 for a $k : (n-k-1)$ split
- Suppose the partition is a $k : (n-k-1)$ split, then $T(n) = T(max(k, n-k-1)) + \Theta(n)$

$$T(n) = \sum_{k=0}^{n-1} X_k \cdot (T(max(k, n-k-1)) + \Theta(n))$$

$$E[T(n)] = E[\sum_{k=0}^{n-1} X_k \cdot (T(max(k, n-k-1)) + \Theta(n))]$$

$$E[T(n)] = \sum_{k=0}^{n-1} E[X_k \cdot (T(max(k, n-k-1)) + \Theta(n))]$$

$$E[T(n)] = \sum_{k=0}^{n-1} E[X_k] \cdot E[(T(max(k, n-k-1)) + \Theta(n))]$$

$$E[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} E[T(max(k, n-k-1))] + \frac{1}{n} \sum_{k=0}^{n-1} \Theta(n)$$

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor \frac{n}{2} \rfloor}^{n-1} E[T(k)] + \Theta(n)$$

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor \frac{n}{2} \rfloor}^{n-1} c \cdot k + d \cdot n$$

$$E[T(n)] \leq \frac{2c}{n}(\frac{3n^2}{8}) + d \cdot n$$

$$E[T(n)] \leq \frac{3c \cdot n}{4} + d \cdot n$$

$$E[T(n)] \leq c \cdot n - (\frac{c \cdot n}{4} - d \cdot n)$$

$$E[T(n)] \leq c \cdot n$$

- We choose $c \geq 4d$

Median of Medians

- Divide array into $\lfloor n/5 \rfloor$ groups of 5 elements each
- Find the median of each 5 element group in constant time
- Let B be the $\lfloor n/5 \rfloor$ medians of each group
- Set $x = B[Select(B, |B|, |B|/2)]$, the median of medians
- Set $k = Partition(A, x)$, let A'/A'' be the elmeents $< x / > x$ respectively
- Similar to quickselect, if $i = k$, return k
- If $i < k$, return $Select(A', k-1, i)$
- If $i > k$, return $Select(A'', n-k, i-k)$
- Since we have at least $3 \cdot \lfloor \frac{n}{10} \rfloor$ elements $\geq x / \leq x$, we recurse on at most $\frac{7n}{10}$ elements and have $T(n) \leq T(n/5) + T(7n/10) + \Theta(n) \Rightarrow T(n) \leq c \cdot n$ which is linear
- In practice it has a high constant which makes it slower than randomised Quickselect
- If we use groups of 3 instead, the time complexity is now $O(n\ log\ n)$

Dynamic Order Statistics

- Order statistics but with updates
- Use BBST augmentation

pls give me A+ prof thanks

pls give me A+ prof thanks

pls give me A+ prof thanks